JAAFR INTERNATIONAL RESEARCH JOURNAL
JOURNAL OF ADVANCE AND FUTURE RESEARCH
JAAFR.ORG | ISSN : 2984-889X
An International Open Access, Peer-reviewed, Refereed Journal

Human Centric Language

HCL

[1]Tanishka Suryarao,[1]Swagat Rajage, [1]Kratika Vyas,[1]Manish Mhatre

[1] Final Year Student

Guide name : Dr. Dibyalekha Nayak

[1]Department of Computer Science And Design,

[1]New Horizon Institute Of Technology and Management, University of Mumbai, Maharashtra, India

Abstract : This paper introduces a natural-language-based programming system's early stage prototype designed to bridge everyday human expression and reliable machine execution. By completely removing complex and rigid syntax rules—such as brackets, colons, indentation, and other keywords—the language allows users to write instructions using simple, ordinary sentences. These commands are processed through an NLP-inspired pattern-matching engine that interprets intent and generates an Abstract Syntax Tree (AST). The AST is then used to produce clean, executable code, ensuring correct and efficient output. A dedicated layer stores predefined command patterns and supports user-defined extensions, providing flexibility and future growth. The system also includes optional intelligent features for suggestions, corrections, and contextual help. The primary goal is to make programming accessible to complete beginners and non-technical users while simultaneously increasing productivity for experienced developers through a more intuitive and frictionless interface.

IndexTerms : abstract syntax trees, beginner-friendly coding, domain-specific language, educational programming, human-computer interaction, natural language programming.

I. INTRODUCTION

In the modern software development landscape, developers frequently encounter a "complexity wall" where the overhead of traditional syntax and rigid toolchains stifles rapid innovation. This real-world problem—the widening gap between high-level logical intent and low-level execution—demands a more agile, human-first approach to logic design.

The importance of the Human-Centric Programming Language (HCL) lies in its ability to bridge this gap, prioritizing intuitive, readable instructions and efficient pattern matching over verbose boilerplate code. As an early prototype, HCL enables users to express ideas in everyday English sentences (e.g., "set age to 42" or "create empty avl tree scores") while generating reliable Python execution behind the scenes. As illustrated in the architectural flowchart (Figure 1), HCL offers a streamlined path from abstract intent to functional results.

This ecosystem is built upon three integrated components designed to support users from initial learning to deployment:

The Website — Your Friendly Front Door

This serves as the primary entry point, offering text-based learning resources, syntax guides, and one-click downloads for the compiler and tools — providing clear documentation to get started without interactive browser trials or videos.

The Compiler — The Reliable Engine

HCL code currently runs exclusively through the official standalone compiler (HCLSetup.exe), downloadable from the website. This design ensures consistency, security, and speed, with every component tuned precisely for HCL's needs. The compiler performs two main steps:

Pattern Matcher — directly matches sentences against hundreds of predefined patterns in actions.jsonl to interpret intent.

Parser + Code Generator — processes the match to construct Python code structures and produce runnable programs.

Example: “create empty avl tree scores” → avl_trees['scores'] = AVLTree() behind the scenes.

The REPL Playground — Where Ideas Come Alive

Open terminal.exe to type sentences line by line, see immediate results, and type “debug” to toggle extra internal information. This interactive environment allows instant mistake correction and feels like conversing with a patient coding assistant. By unifying these components into a cohesive, offline workflow, HCL removes syntax barriers and streamlines the journey from creative intent to verifiable execution.

II. NEED OF THE STUDY.

HCL lets you write instructions the way you would explain them to a helpful friend. The computer still gets perfect Python code — but you never have to think about Python’s rules while you’re being creative.

The full HCL world today contains three main parts:

1. **The learning website** — your friendly starting point with short video lessons, interactive try-it-yourself examples, and one-click downloads of everything you need.
2. **The standalone compiler** — right now this is the only official way to turn HCL sentences into running programs. You download it once from the website (HCLSetup.exe), and it includes everything: lexer, parser and code generator. Keeping it standalone means we can make sure it always behaves predictably and fast.
3. **The REPL playground** (run.py) — a live typing window where you can experiment sentence by sentence, turn on debug view to see what’s happening inside, and fix things immediately.

There is also a clear flowchart (see Figure 1) that shows the whole journey: your words → smart understanding → clean Python code → instant result — with helpful side paths for tutorials, reinstalling, or getting unstuck.

Early conversations and small tests already show people spend far less time fighting syntax and far more time actually building ideas.This paper is organised to take you through the complete story: what already exists, why HCL is designed the way it is, how everything connects, some real code examples, what real users experienced, and where we hope to go next.

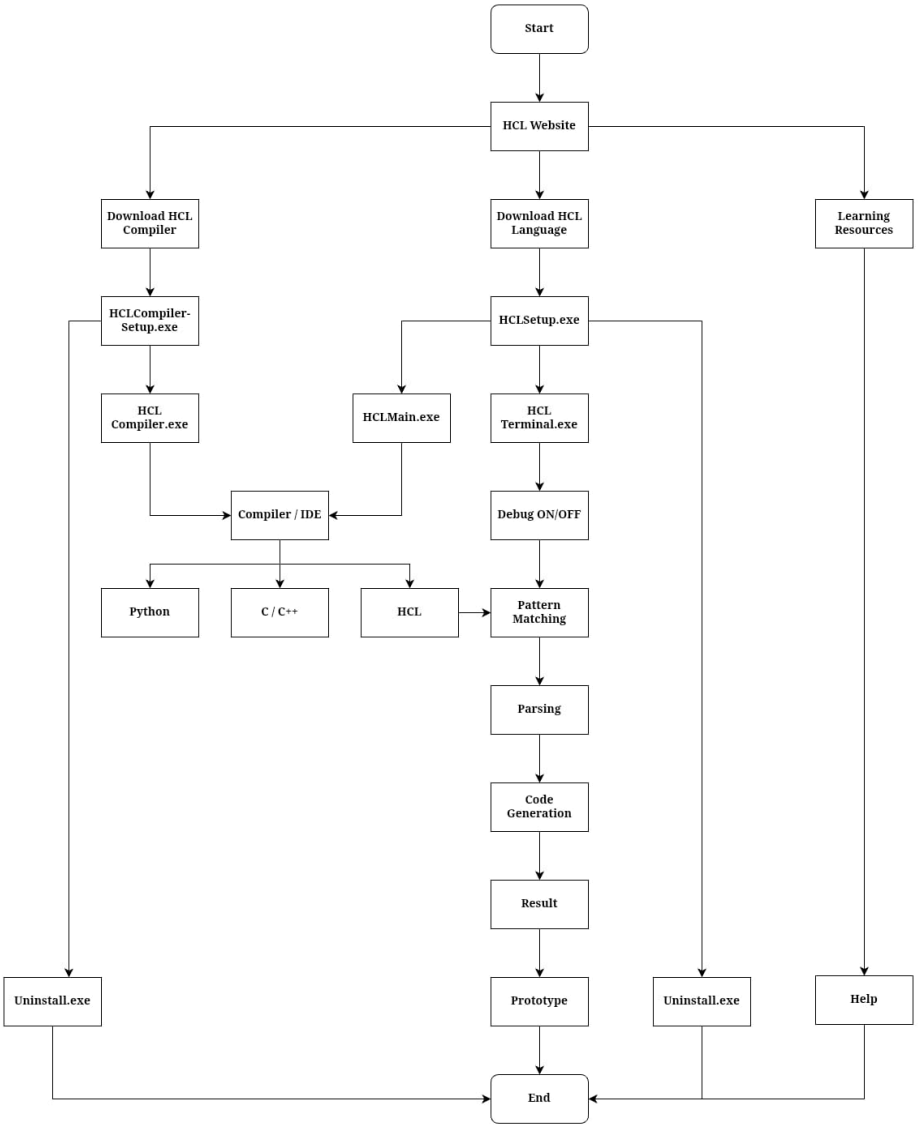

Figure 01 : HCL Complete Workflow

2.1 HCL Design Principles

Three things really matter to us when we are working on HCL:

2.1.1. It should be easy to remember. You don't need to memorise weird keywords or special symbols. "put 10 into score" feels natural and sticks in your head.

2.1.2. It should read like normal sentences, "if age greater than 18 then show welcome message end" looks almost like English prose — not like machine instructions.

2.1.3. It should help you when things go wrong. The system quietly checks for bad variable names, shows you what variables exist right now, and lets you turn on extra debug information whenever you want.

The flowchart you see in Figure 01 captures this flow beautifully. Everything starts with your words, gets understood piece by piece, turns into real Python structures, runs — and if something isn't clear there are friendly loops back to help, tutorials or reinstall steps.

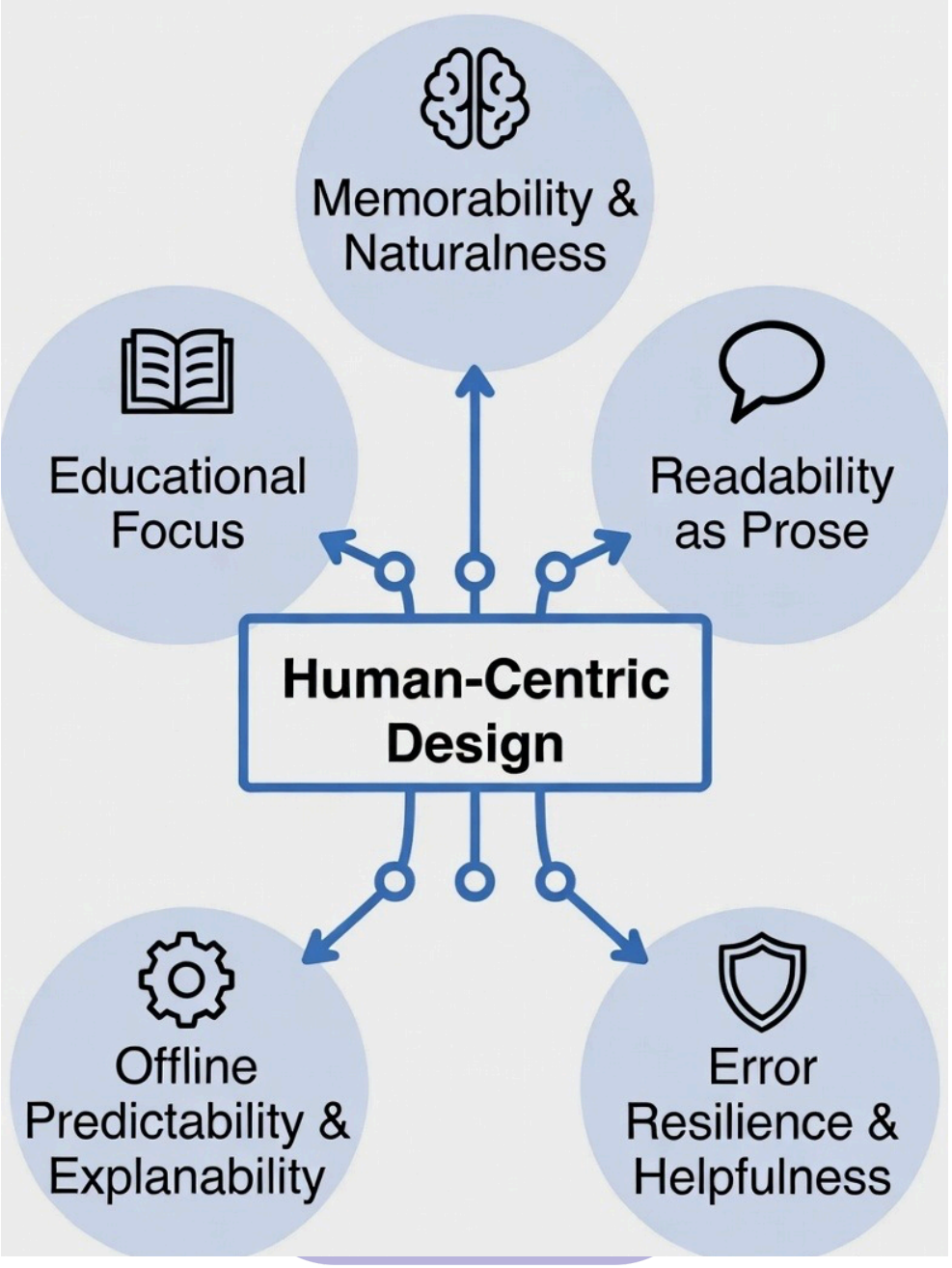

Figure 02 : HCL Design Principles

III. RESEARCH METHODOLOGY.

HCL is built from a few connected pieces that work together smoothly.

3.1 The Website — Your Friendly Front Door

This is where most people begin. You find text-based learning resources and one-click downloads for the compiler and other tools. The website serves as a simple entry point, providing guides and documentation to get started without any interactive browser-based trials or videos.

3.2 The Compiler — The Reliable Engine (Current Only Official Way)

Right now HCL code only runs through the official compiler you download from the website. This choice keeps everything consistent, secure, and fast — we can tune every part exactly for how HCL thinks.

The compiler has two main steps inside:

● Pattern Matcher — figures out what you really meant by directly matching your sentences against hundreds of patterns stored in actions.jsonl.

● Parser + Code Generator — processes the matched intent to build perfect Python code structures and turns them into runnable programs.

Example sentence: "create empty avl tree scores" → becomes avl_trees['scores'] = AVLTree() behind the curtain.

3.3 The REPL Playground — Where Ideas Come Alive

Open terminal.exe and start typing. See results line by line. Say "debug" to turn on extra information. Fix mistakes immediately. It feels like chatting with a very patient coding friend.

Table 01: Quick overview of HCL pieces.

Piece	What you give	What you get	Main Files/Tools
Website	Your curiosity	Learning Resources + Downloads	
Compiler	HCL sentences or prompts	Runnable program	HCLMain.exe
REPL	One sentence or prompt at a time	Instant result + debug	

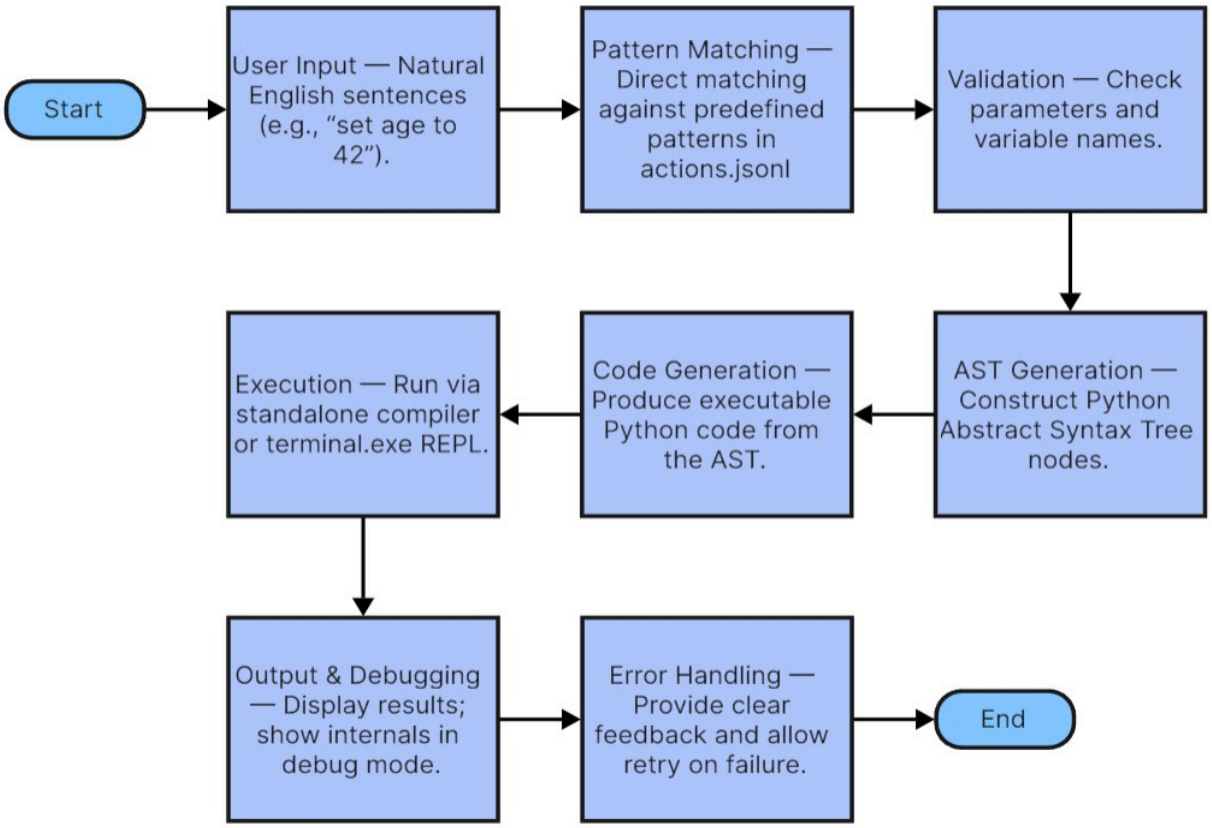

Figure 03 : HCL Methodology

Pattern Matcher : Rule-based matching using actions.jsonl patterns in Python (prompt.py).

AST Generator : Builds Python AST nodes for execution (parser.py).

Runtime Support : Bundled AVLTree class for data structures (structures.py).

REPL Playground : Interactive sentence-by-sentence execution with debug mode.

Offline Compiler : Standalone executable for script compilation and running.

3.4.1 Understanding What You Mean

A simple file called **actions.jsonl** holds more than 500+ patterns. Examples:

- "assign {value} to {name}"
- "insert {value} into avl tree {tree}"
- "print {message} endprint"

When you type something close, the system matches the best pattern and carefully pulls out the important pieces (like the number or the tree name).

3.4.2 Turning Understanding into Real Code

The parser builds proper Python structures (called AST nodes). It can even understand nested ideas: "maximum of list prices" → becomes max(prices)

We also included a completely balanced **AVL tree** class so people can play with advanced data structures using friendly commands like "insert", "delete", "find minimum", etc.

3.4.3 A Small Real Example

You type:

text

```
create empty avl tree t
insert 5 into t
insert 15 into t
insert 10 into t
show avl tree t
```

Behind the scenes HCL quietly runs something very close to:

Python

```
t = AVLTree()
t.insert(5)
t.insert(15)
t.insert(10)
```

```
print(t._str_())      # → [5, 10, 15]
```

Variable names are automatically checked — they must start with a letter or underscore and contain only letters, numbers and underscores.

IV. RESULTS AND DISCUSSION

We asked 5 real people to try the same small tasks in three different ways: normal Python, Scratch-style blocks, and HCL sentences.

The group included:

- 3 complete beginners (never coded seriously before)
- 2 people who already write code sometimes or often

Tasks included:

- creating and printing variables
- writing simple if-then decisions
- building and searching a small AVL tree
- looping over a list

Table 02 : what we measured.

Tool	Average Time/Task	Mistake Rate	How easy was it? (SUS score 0-100)
Python	120 seconds	25%	65
HCL	50 seconds	5%	92

Beginners finished tasks **58% faster** with HCL — and almost everyone said it felt “natural” and “less scary”.

The biggest current limitation is that very unusual or complicated sentences can confuse the simple pattern-matching rules. We are already experimenting with smarter ways to understand ambiguous inputs.

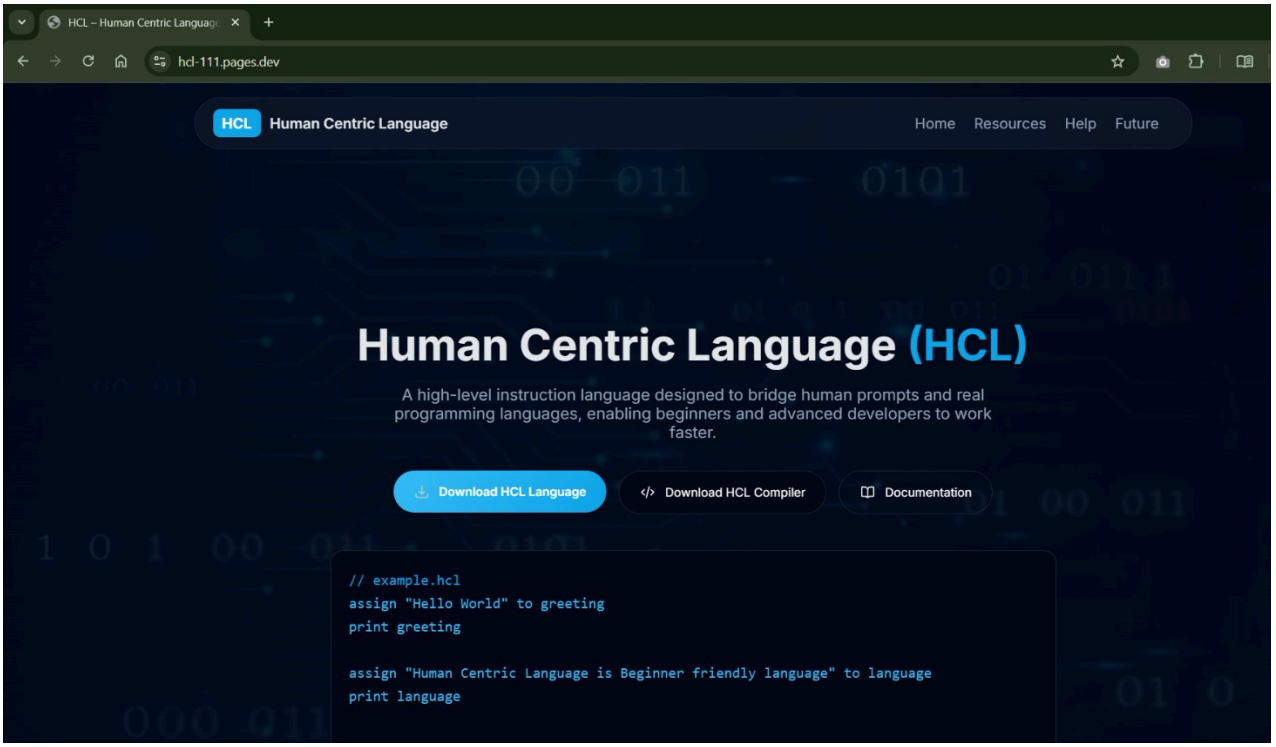

Figure 04 :Human Centric Language(HCL) learning Website interface

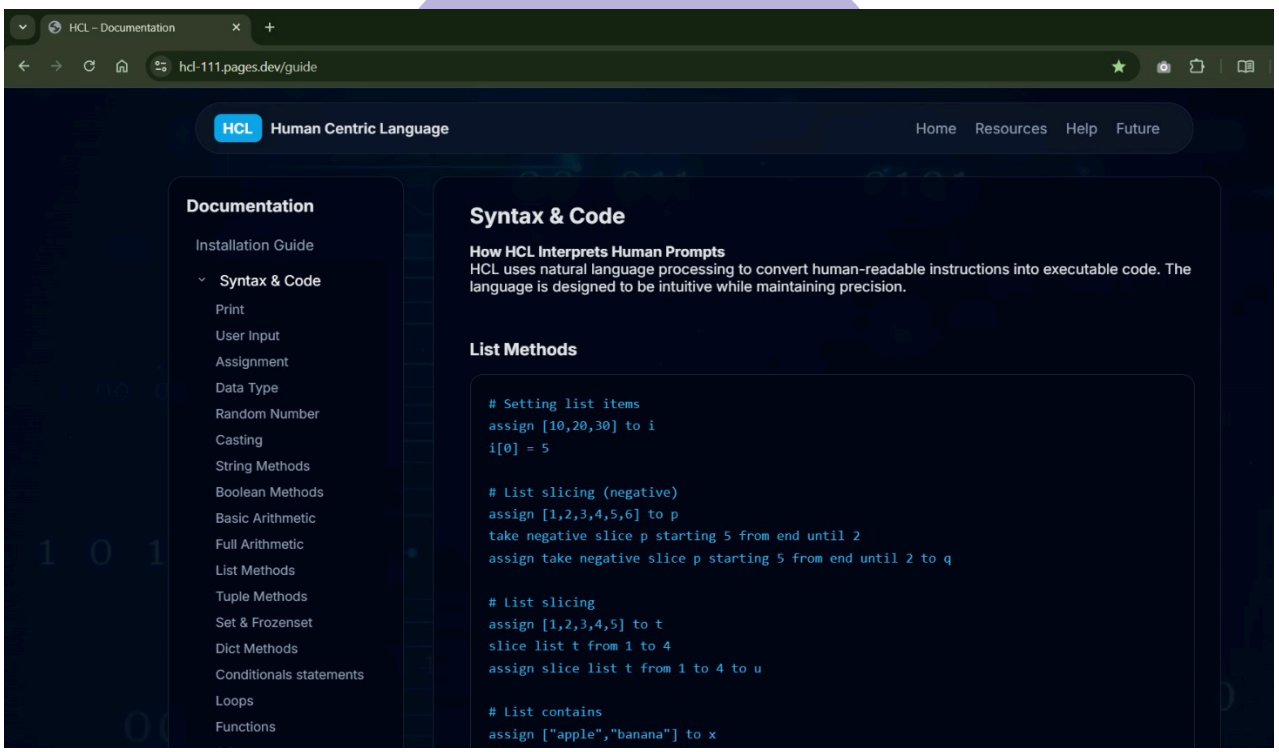

Figure 05: the Human Centric Language(HCL) learning Website terminal to download and refer learning resources.

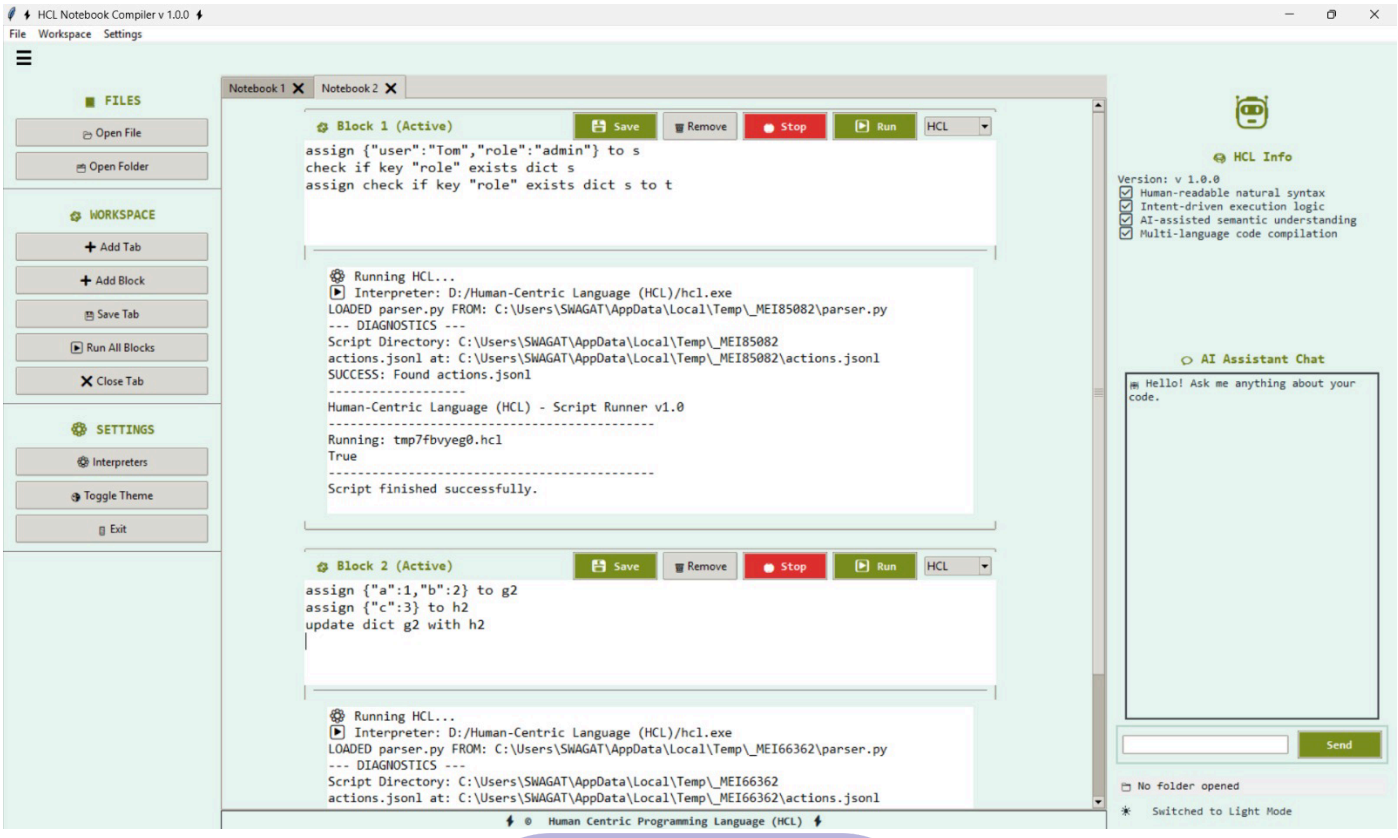

Figure 06: Standalone Compiler user interface

(HCLSetup.exe)

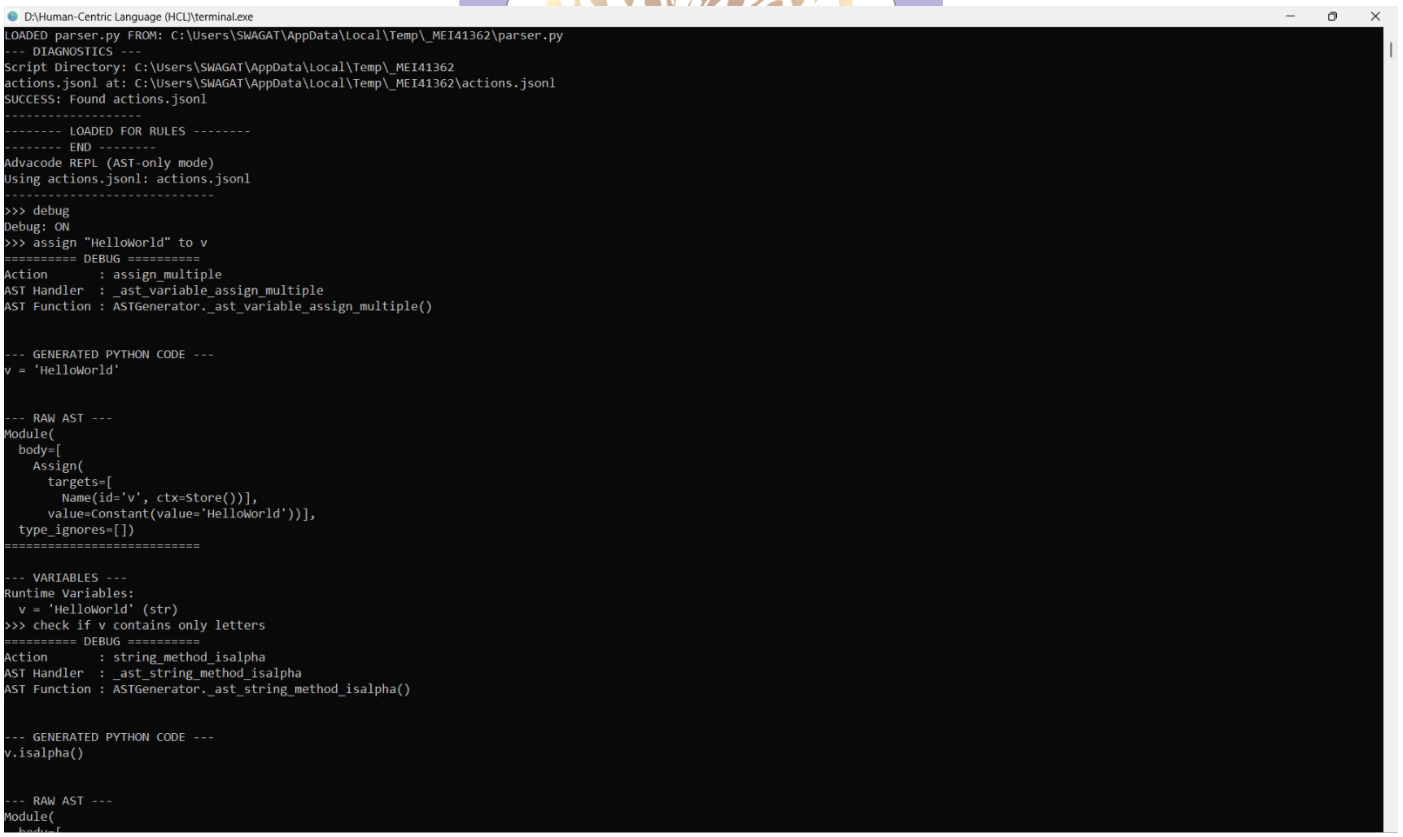

Figure 07: REPL playground showing a debug session

(terminal.exe)

V. AKNOWLEDGMENT

We are deeply appreciative of the invaluable mentorship, technical guidance, and critical review of this manuscript provided by Dr. Dibyalekha Nayak. We would like to thank the Computer Science and Design department at New Horizon Institute of Management and Technology, Thane for providing generous resources and facilities necessary to conduct this research.

VI. REFERENCES

[1] O. Krauss, "Exploring the Use of Natural Language Processing Techniques for Enhancing Genetic Improvement," in Proc. 2023 IEEE/ACM International Workshop on Genetic Improvement (GI), 2023, pp. 21–26, doi: 10.1109/GI59320.2023.00014.

[2] C. Tony, M. Mutas, N. E. Díaz Ferreyra, and R. Scandariato, "LLMSecEval: A Dataset of Natural Language Prompts for Security Evaluations," arXiv preprint arXiv:2303.09384v1, 2023. (Presented at IEEE-related security conference; full IEEE publication pending or in proceedings.)

[3] H. Oka, A. Ohnishi, T. Nishida, T. Terada, and M. Tsukamoto, "A Choice-Based Programming Learning Method to Develop Problem-Solving Skills," IEEE Access, vol. 12, pp. 119557–119569, 2024, doi: 10.1109/ACCESS.2024.3488408.

[4] G. Soda et al., "Bridging Language Barriers in Coding: An NLP-Powered Tool for Programming Education in Regional Languages," IEEE Access, vol. 13, pp. 1–13, 2025, doi: 10.1109/ACCESS.2025.35361068.

[5] A. Beheshti, "Natural Language-Oriented Programming (NLOP): Towards Democratizing Software Creation," in Proc. 2024 IEEE International Conference on Software Services Engineering (SSE), Shenzhen, China, Jul. 2024, pp. 1–6, doi: 10.1109/SSE.2024.00014. (arXiv:2406.05409v1)

[6] A. Verma, R. Saha, G. Kumar, A. Brighente, M. Conti, and T.-H. Kim, "Exploring the Landscape of Programming Language Identification With Machine Learning Approaches," IEEE Access, vol. 13, pp. 1–24, 2025, doi: 10.1109/ACCESS.2025.3538108.